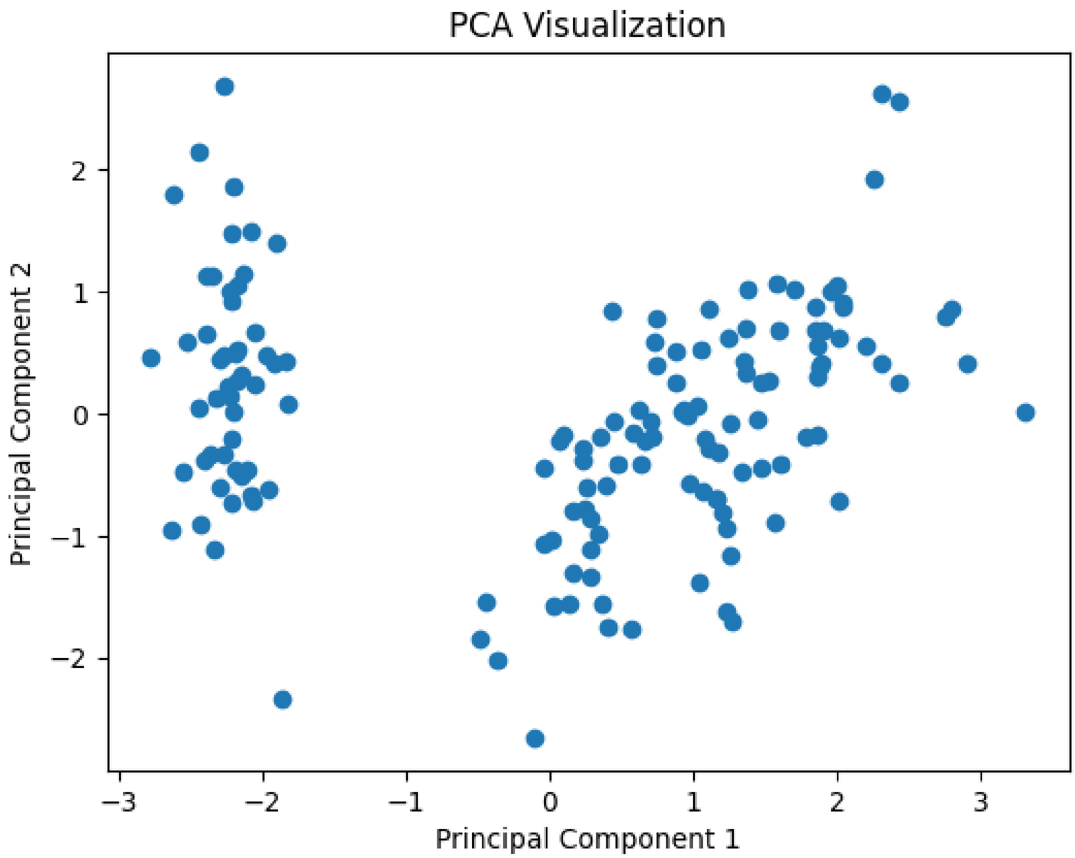
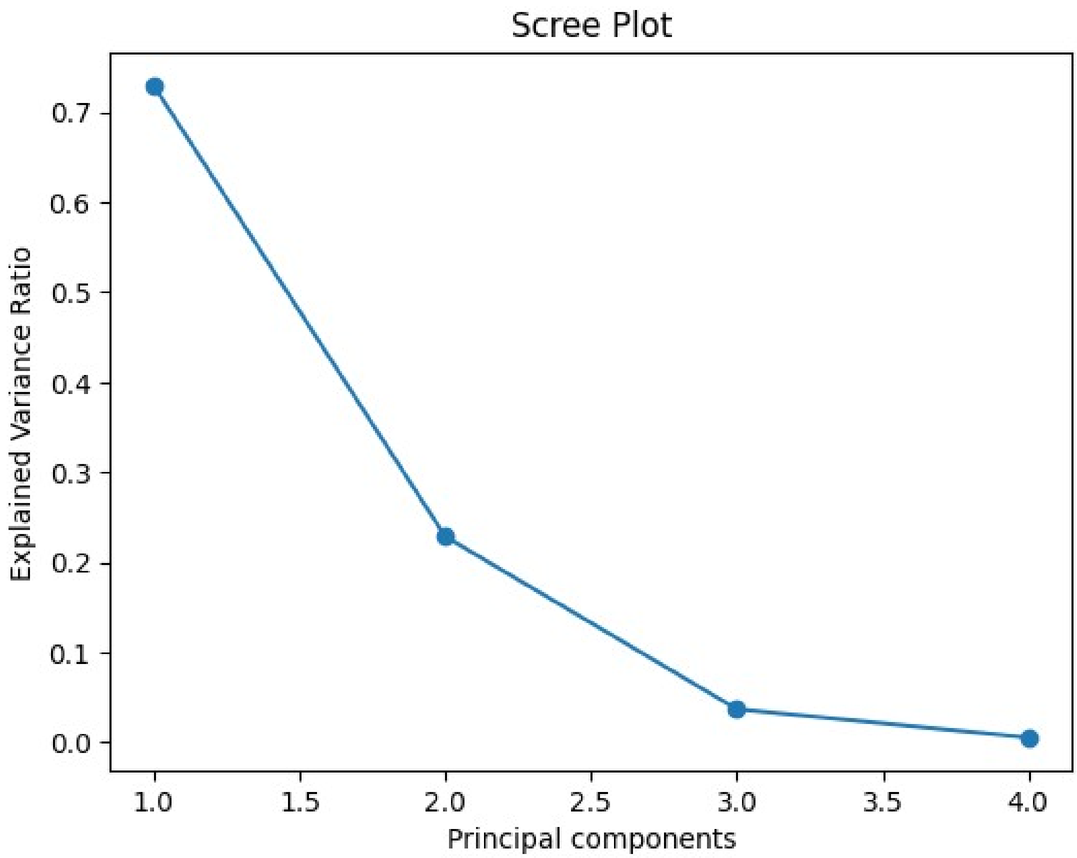
PCA (Principal Component Analysis)

```python
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import pandas as pd
# Load dataset
iris = load_iris()
X = iris.data
# Standardize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
# Explained Variance Ratio
print("Explained Variance Ratio:")
print(pca.explained_variance_ratio_)
# PCA DataFrame
df = pd.DataFrame(
    X_pca,
    columns=["PC1", "PC2"]
)
print(df.head())
# Scatter Plot
plt.scatter(X_pca[:,0], X_pca[:,1])
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Visualization")
plt.show()
```

```
Explained Variance Ratio:
[0.72962445 0.22850762]
        PC1       PC2
0 -2.264703  0.480027
1 -2.080961 -0.674134
2 -2.364229 -0.341908
3 -2.299384 -0.597395
4 -2.389842  0.646835
```




```
#scree plot
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
pca=PCA()
pca.fit(X_scaled)
plt.plot(
    range(1,len(pca.explained_variance_ratio_)+1),
    pca.explained_variance_ratio_,
    marker='o'
)
plt.xlabel("Principal components")
plt.ylabel("Explained Variance Ratio")
plt.title("Scree Plot")
plt.show()
```




```
#K-Fold Cluster
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import KFold, cross_val_score
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
# Model
model = DecisionTreeClassifier()
# K-Fold
kf = KFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)
scores = cross_val_score(
    model,
    X,
    y,
    cv=kf
)
```

```
print("Scores:", scores)
print("Mean Accuracy:", scores.mean())
```

```
Scores: [1.         0.96666667 0.93333333 0.93333333 0.93333333]
Mean Accuracy: 0.9533333333333335
```

```
#stratiedKFold
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
# Model
model = DecisionTreeClassifier()
# Stratified K-Fold
skf = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)
scores = cross_val_score(
    model,
    X,
    y,
    cv=skf
)
print("Scores:", scores)
print("Mean Accuracy:", scores.mean())
```

```
Scores: [1.         0.96666667 0.93333333 0.96666667 0.9        ]
Mean Accuracy: 0.9533333333333335
```

Hyperparameter Tuning

```
## GridSearchCV code:
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV
#Dataset
iris = load_iris()
x = iris.data
y = iris.target
#Model
model = DecisionTreeClassifier()
# parameter grid
param_grid = {
```

```
    'max_depth' : [2,3,4,5],
    'min_samples_split' : [2,4,6]}
#Grid Search
grid = GridSearchCV(
    model,
    param_grid,
    cv=5,
    scoring='accuracy'
)
grid.fit(x,y)
print("Best Parameters:",grid.best_params_)
print("Best Score:",grid.best_score_)
```

```
Best Parameters: {'max_depth': 3, 'min_samples_split': 4}
Best Score: 0.9733333333333334
```

```
#RandomizedSearchCV code:
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import RandomizedSearchCV
#Dataset
iris =load_iris()
x=iris.data
y=iris.target
#model
model=DecisionTreeClassifier()
#parameter grid
param_dist={
    'max_depth':[2,3,4,5,6,7,8],
    'min_samples_split':[2,4,6,8,10]}
# randomized Search
random_search=RandomizedSearchCV(
    model,
    param_distributions=param_dist,
    n_iter=10,
    cv=5,
    random_state=42
)
random_search.fit(x,y)
print("Best Parameters:",random_search.best_params_)
print("Best Scores:",random_search.best_score_)
```

```
Best Parameters: {'min_samples_split': 8, 'max_depth': 3}
Best Scores: 0.9733333333333334
```

```
!pip install Optuna
```

```
Collecting Optuna
  Downloading optuna-4.9.0-py3-none-any.whl.metadata (15 kB)
Collecting alembic>=1.5.0 (from Optuna)
```

```
  Downloading alembic-1.18.4-py3-none-any.whl.metadata (7.2 kB)
Collecting colorlog (from Optuna)
  Downloading colorlog-6.10.1-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: numpy in c:\users\monik\appdata\
roaming\python\python313\site-packages (from Optuna) (2.4.2)
Requirement already satisfied: packaging>=20.0 in c:\users\monik\
appdata\roaming\python\python313\site-packages (from Optuna) (26.0)
Requirement already satisfied: sqlalchemy>=1.4.2 in c:\users\monik\
appdata\roaming\python\python313\site-packages (from Optuna) (2.0.48)
Requirement already satisfied: tqdm in c:\users\monik\anaconda3\lib\
site-packages (from Optuna) (4.67.1)
Requirement already satisfied: PyYAML in c:\users\monik\anaconda3\lib\
site-packages (from Optuna) (6.0.3)
Collecting Mako (from alembic>=1.5.0->Optuna)
  Downloading mako-1.3.12-py3-none-any.whl.metadata (2.9 kB)
Requirement already satisfied: typing-extensions>=4.12 in c:\users\
monik\appdata\roaming\python\python313\site-packages (from
alembic>=1.5.0->Optuna) (4.15.0)
Requirement already satisfied: greenlet>=1 in c:\users\monik\appdata\
roaming\python\python313\site-packages (from sqlalchemy>=1.4.2-
>Optuna) (3.3.2)
Requirement already satisfied: colorama in c:\users\monik\appdata\
roaming\python\python313\site-packages (from colorlog->Optuna) (0.4.6)
Requirement already satisfied: MarkupSafe>=0.9.2 in c:\users\monik\
appdata\roaming\python\python313\site-packages (from Mako-
>alembic>=1.5.0->Optuna) (3.0.3)
Downloading optuna-4.9.0-py3-none-any.whl (425 kB)
Downloading alembic-1.18.4-py3-none-any.whl (263 kB)
Downloading colorlog-6.10.1-py3-none-any.whl (11 kB)
Downloading mako-1.3.12-py3-none-any.whl (78 kB)
Installing collected packages: Mako, colorlog, alembic, Optuna

   ---------------------------------------- 0/4 [Mako]
   ---------------------------------------- 0/4 [Mako]
   -------------------- ------------------- 2/4 [alembic]
   -------------------- ------------------- 2/4 [alembic]
   -------------------- ------------------- 2/4 [alembic]
   -------------------- ------------------- 2/4 [alembic]
   -------------------- ------------------- 2/4 [alembic]
   ------------------------------ --------- 3/4 [Optuna]
   ------------------------------ --------- 3/4 [Optuna]
   ------------------------------ --------- 3/4 [Optuna]
   ------------------------------ --------- 3/4 [Optuna]
   ------------------------------ --------- 3/4 [Optuna]
   ------------------------------ --------- 3/4 [Optuna]
   ------------------------------ --------- 3/4 [Optuna]
   ------------------------------ --------- 3/4 [Optuna]
   ------------------------------ --------- 3/4 [Optuna]
   ------------------------------ --------- 3/4 [Optuna]
```

```
   ------------------------------ --------- 3/4 [Optuna]
   ------------------------------ --------- 3/4 [Optuna]
   ------------------------------ --------- 3/4 [Optuna]
   ------------------------------ --------- 3/4 [Optuna]
   ---------------------------------------- 4/4 [Optuna]

Successfully installed Mako-1.3.12 Optuna-4.9.0 alembic-1.18.4
colorlog-6.10.1
```

```
#Optuna Basic code:
import optuna
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
#Dataset
iris =load_iris()
x=iris.data
y=iris.target
#Objective function
def objective(trail):
    max_depth=trail.suggest_int(
        "max_depth",2,10)
    min_samples_split=trail.suggest_int(
        "min_samples_split",2,10)

    model=DecisionTreeClassifier(
        max_depth=max_depth,
        min_samples_split=min_samples_split)
    score=cross_val_score(model,X,y,cv=5).mean()
    return score
#Study
study=optuna.create_study(direction="maximize")
study.optimize(objective,n_trials=20)
print("Best Parameters:")
print(study.best_params)
print("Best Scores:")
print(study.best_value)
```

```
[I 2026-06-12 12:47:34,883] A new study created in memory with name:
no-name-d4b8a859-2305-4c62-8829-eb717fbbc6ed
[I 2026-06-12 12:47:34,913] Trial 0 finished with value:
0.9666666666666668 and parameters: {'max_depth': 10,
'min_samples_split': 7}. Best is trial 0 with value:
0.9666666666666668.
[I 2026-06-12 12:47:34,936] Trial 1 finished with value:
0.9666666666666668 and parameters: {'max_depth': 4,
'min_samples_split': 10}. Best is trial 0 with value:
0.9666666666666668.
[I 2026-06-12 12:47:34,957] Trial 2 finished with value:
```

```
0.9666666666666668 and parameters: {'max_depth': 10,
'min_samples_split': 10}. Best is trial 0 with value:
0.9666666666666668.
[I 2026-06-12 12:47:34,976] Trial 3 finished with value:
0.9333333333333332 and parameters: {'max_depth': 2,
'min_samples_split': 8}. Best is trial 0 with value:
0.9666666666666668.
[I 2026-06-12 12:47:34,998] Trial 4 finished with value:
0.9533333333333334 and parameters: {'max_depth': 4,
'min_samples_split': 3}. Best is trial 0 with value:
0.9666666666666668.
[I 2026-06-12 12:47:35,019] Trial 5 finished with value:
0.9533333333333334 and parameters: {'max_depth': 9,
'min_samples_split': 4}. Best is trial 0 with value:
0.9666666666666668.
[I 2026-06-12 12:47:35,041] Trial 6 finished with value:
0.9666666666666668 and parameters: {'max_depth': 7,
'min_samples_split': 5}. Best is trial 0 with value:
0.9666666666666668.
[I 2026-06-12 12:47:35,065] Trial 7 finished with value: 0.96 and
parameters: {'max_depth': 3, 'min_samples_split': 6}. Best is trial 0
with value: 0.9666666666666668.
[I 2026-06-12 12:47:35,087] Trial 8 finished with value:
0.9333333333333332 and parameters: {'max_depth': 2,
'min_samples_split': 7}. Best is trial 0 with value:
0.9666666666666668.
[I 2026-06-12 12:47:35,106] Trial 9 finished with value:
0.9666666666666668 and parameters: {'max_depth': 8,
'min_samples_split': 9}. Best is trial 0 with value:
0.9666666666666668.
[I 2026-06-12 12:47:35,175] Trial 10 finished with value:
0.9666666666666668 and parameters: {'max_depth': 6,
'min_samples_split': 6}. Best is trial 0 with value:
0.9666666666666668.
[I 2026-06-12 12:47:35,200] Trial 11 finished with value:
0.9666666666666668 and parameters: {'max_depth': 5,
'min_samples_split': 10}. Best is trial 0 with value:
0.9666666666666668.
[I 2026-06-12 12:47:35,228] Trial 12 finished with value:
0.9666666666666668 and parameters: {'max_depth': 10,
'min_samples_split': 8}. Best is trial 0 with value:
0.9666666666666668.
[I 2026-06-12 12:47:35,260] Trial 13 finished with value:
0.9600000000000002 and parameters: {'max_depth': 7,
'min_samples_split': 2}. Best is trial 0 with value:
0.9666666666666668.
[I 2026-06-12 12:47:35,316] Trial 14 finished with value:
0.9666666666666668 and parameters: {'max_depth': 5,
'min_samples_split': 8}. Best is trial 0 with value:
```

```
0.9666666666666668.
[I 2026-06-12 12:47:35,343] Trial 15 finished with value:
0.9666666666666668 and parameters: {'max_depth': 8,
'min_samples_split': 9}. Best is trial 0 with value:
0.9666666666666668.
[I 2026-06-12 12:47:35,371] Trial 16 finished with value:
0.9666666666666668 and parameters: {'max_depth': 4,
'min_samples_split': 6}. Best is trial 0 with value:
0.9666666666666668.
[I 2026-06-12 12:47:35,395] Trial 17 finished with value:
0.9666666666666668 and parameters: {'max_depth': 6,
'min_samples_split': 10}. Best is trial 0 with value:
0.9666666666666668.
[I 2026-06-12 12:47:35,426] Trial 18 finished with value:
0.9666666666666668 and parameters: {'max_depth': 9,
'min_samples_split': 4}. Best is trial 0 with value:
0.9666666666666668.
[I 2026-06-12 12:47:35,454] Trial 19 finished with value:
0.9666666666666668 and parameters: {'max_depth': 4,
'min_samples_split': 7}. Best is trial 0 with value:
0.9666666666666668.

Best Parameters:
{'max_depth': 10, 'min_samples_split': 7}
Best Scores:
0.9666666666666668
```